결제 운영 신뢰성 작업 보고서 (ISE-40·41·42·44)

Nov 3, 2026 · @Junhyun An

한눈에 보기

결제 M3의 운영 신뢰성 범위(ISE-40·41·42·44)를 PR 12개로 끝내고 dev에 머지했다. 목표는 장애가 나도 돈이 두 번 빠지거나 사라지지 않고, 사람이 그 사실을 알아채고, 안전하게 고칠 수 있게 하는 것이다.

네 묶음은 차례로 쌓인다. 먼저 시스템이 스스로 다시 시도한다(이벤트 신뢰성). 스스로 못 고치면 알린다(관측). 알림을 받은 사람은 정해진 명령으로만 고친다(운영자 복구). 마지막으로 이 전부가 실제로 지켜지는지 장애를 일부러 일으켜 확인한다(장애 시나리오 테스트).



이번 범위의 네 겹 · 자동 복구, 관측, 운영자 복구, 증명

아래로 흐르는 화살표가 "못 푼 일이 사람에게 가는 길"이고, 왼쪽으로 도는 화살표가 "사람이 되돌린 일을 다시 자동 복구가 끝내는 길"이다. 바깥 점선은 이 전부를 장애 테스트가 감싼다는 뜻이다.

묶음	이슈	PR	한 일
이벤트 신뢰성	ISE-145	#229	이벤트 발행 재시도(간격을 늘려 가며), 영구 오류는 `FAILED` 로 정지
이벤트 신뢰성	ISE-155	#248	긴급 수정: 팀원 PR과 겹친 DB 마이그레이션 V5를 V6으로
이벤트 신뢰성	ISE-27	#225	소비 실패 시 재시도 뒤 DLQ로 보내는 공통 부품
이벤트 신뢰성	ISE-146	#251	오래된 Outbox·처리 이력·멱등 기록 정리
관측	ISE-156	#256	결제 지표, DB 집계 Gauge, PG 보호 장치 지표
관측	ISE-157	#257	Prometheus 알림 21개와 규칙 테스트, Grafana 대시보드
운영자 복구	ISE-158	#258	명령 실행 기반, 감사 테이블, `inspect` · `outbox requeue`
운영자 복구	ISE-159	#259	`payment resolve` · `refund resolve`
운영자 복구	ISE-160	#262	`refund retry` · `dead-letter resolve`
장애 테스트	ISE-128	#263	Mock PG 호출 수 기록, 원장 조회, 환불 장애 주입
장애 테스트	ISE-161	#264	검증기 V1~V3, FT-1 중복·FT-2 응답 유실
장애 테스트	ISE-162	#266	FT-3 재시작·FT-4 DB 장애·FT-5 환불 중단·금액 불일치

부모 이슈 ISE-40·41·42·44를 닫았고, 하위 이슈마다 완료 기준을 PR 번호와 확인 방법으로 채웠다. 새로 드러난 문제는 후속 이슈 ISE-163으로 만들었다.

바탕 개념: 결제에서 "정확히 한 번"이 어려운 이유

결제 한 건은 우리 DB, 외부 PG, Kafka 세 곳에 걸쳐 있고, 셋을 한 트랜잭션으로 묶을 방법은 없다. 그래서 어느 자리에서 끊겨도 "다시 해도 같은 결과"가 나오게 만들고, 다시 하는 주체를 따로 둔다. 이번 범위는 그 "다시 하는 주체"를 운영 수준으로 완성한 일이다.

승인 한 건의 흐름

1. TX1: 결제 행을 `REQUESTED` 로 만들고 멱등 기록을 남긴 뒤 커밋한다.
2. PG 승인 호출. 트랜잭션 밖이다. DB 연결을 쥐고 외부 응답을 기다리면 폭주 때 커넥션 풀이 바닥나기 때문이다.
3. TX2: PG 결과를 기록한다. 승인이면 `APPROVED` 와 `PaymentApproved` 이벤트(Outbox 행)를 같은 트랜잭션에 쓴다.
4. 릴레이가 Outbox 행을 Kafka로 보내고, 예매 서비스가 받아 예매를 확정한다.

끊기는 자리와 남는 상태

끊긴 자리	남는 상태	누가 고치나
TX1 뒤, PG 호출 전	결제 `REQUESTED`, PG에는 주문 없음	재확인이 PG `NotFound`를 보고 늦은 승인 방지(망취소) 뒤 `DECLINED(NOT_SENT)`
PG 호출 중(응답 유실)	`UNKNOWN`. PG에는 승인됐을 수도 있음	재확인이 PG 조회 결과대로 확정
TX2 뒤, Kafka 전송 전	`APPROVED`, Outbox 행 대기	릴레이 재시도(이번 ISE-145)
전송 뒤 확인 전	같은 이벤트가 두 번 나갈 수 있음	받는 쪽이 `eventId`로 거름
어느 자동 경로도 못 푼 경우	`NEEDS_MANUAL_CHECK`(NMC), DLQ OPEN 등	알림 → 운영자 명령(이번 ISE-41·42)

세 가지 기본 도구

- **멱등(idempotency)**: 같은 요청을 여러 번 해도 한 번 한 것과 결과가 같다. 클라이언트 멱등키, PG 멱등키(`orderId`, `pg_refund_id`), DB `UNIQUE` 제약이 각각 한 겹이다.
- **최소 한 번 전달(at-least-once)과 중복 제거**: 잃지 않는 대신 중복을 허용하고, 받는 쪽이 거른다. "정확히 한 번"은 이 둘의 결합으로 만든다.
- **명시적 미해결**: 모르는 상태를 성공이나 실패로 추측하지 않는다. `UNKNOWN`·NMC로 남겨 지표에 잡히게 한다.

이번 범위가 지키는 약속(불변식)

- V1 승인 중복 없음: 예매 하나에 결제는 1건 이하, PG 승인 호출도 1번 이하.
- V2 환불 중복 없음: 결제 하나에 환불은 1건 이하, PG 환불 합계는 승인 금액 이하.
- V3 사라지는 돈 없음: 모든 결제는 정상 종결이거나, 지표에 잡힌 미해결이다.

아래 네 묶음은 모두 이 세 약속을 지키기 위한 것이고, 마지막 장애 테스트는 이 세 가지를 그대로 검사한다.

이벤트 신뢰성 (ISE-40)

이벤트는 이제 보내는 쪽에서도 받는 쪽에서도 잃지 않는다. 다만 영원히 못 가는 이벤트는 멈춰 세우고 사람을 부른다. compose에서 Kafka를 실제로 멈춘 채 승인했더니 승인 응답은 200이었고, Outbox는 5분 동안 13번 재시도했다. Kafka를 다시 켜자 25초 안에 발행됐다.

보내는 쪽: Outbox 재시도 (#229, ISE-145)

개념. Transactional Outbox는 상태 변경과 "보낼 이벤트"를 같은 DB 트랜잭션에 쓰고, 별도 릴레이가 그 행을 읽어 Kafka로 보내는 방식이다. 커밋된 이벤트는 언젠가 반드시 나간다. 문제는 "언제가"다. 실패한 행을 어떻게 다시 보낼지가 이번 내용이다.

- **일시 오류와 영구 오류를 가른다.** 브로커 다운은 기다리면 풀린다. 직렬화 실패·메시지 크기 초과·잘못된 토픽은 백 번 보내도 같다. 일시 오류는 끊임없이 재시도하고, 영구 오류만 `FAILED`로 멈춘다.
- **모르는 예외는 일시 오류로 본다.** 모르는 예외를 영구로 보면 멀엉한 이벤트가 멈춰 돈이 묶인다. 반대로 일시로 보면 최악이 "계속 재시도 + 지연 알림"이다.
- **지수 백오프와 흔들림(jitter).** 실패할수록 간격을 두 배로 늘리고(500ms에서 시작, 상한 60초) 무작위 흔들림을 섞는다. 흔들림이 없으면 같이 실패한 행들이 같은 순간에 몰려든다. equal jitter를 써서 실제 대기는 기본 지연의 50~100%다. 0에 가까운 재시도가 나오지 않는다.
- **같은 키의 순서를 지킨다.** 같은 예매의 이벤트는 순서가 중요하다(승인 뒤 환불). 앞 행이 `FAILED`면 같은 키의 뒤 행도 집지 않는다. 운영자가 원인을 고치고 `outbox requeue`로 되살려야 뒤 행이 이어 나간다.
- **전용 실행기.** 릴레이는 다른 배경 작업과 스레드를 나눠 쓴다. 느린 정리 작업이 이벤트 발행을 기다리게 하지 않기 위해서다.

코드 위치: `common`의 `OutboxBackoff`, `OutboxFailureClassifier`, `OutboxRepository.claimNext`. 예매 서비스도 같은 부품을 쓴다.

받는 쪽: 재시도와 DLQ (#225, ISE-27)

개념. DLQ(Dead Letter Queue)는 계속 실패하는 메시지 하나가 파티션 전체를 막지 않게, 몇 번 재시도한 뒤 별도 토픽(`<토픽>.dlq`)으로 치워 두는 곳이다.

상황	처리	이유
일시 실패	1·2·4초 간격 재시도 뒤 DLQ	잠깐 장애는 재시도로 풀리고, 오래가면 뒤 메시지를 막지 않는다
내용이 틀린 메시지	재시도 없이 바로 DLQ	다시 해도 같다
DLQ로 보내기 실패	오프셋을 커밋하지 않고 다시 처리	건너뛰면 메시지를 잃는다
DLQ를 읽는 처리기의 실패	최대 60초 간격으로 끝없이 재시도	더 갈 곳이 없다. 건너뛰면 환불 의무가 사라진다
추첨 자동결제가 PG 호출 허가를 못 얻음	DLQ로 보내지 않고 1초마다 같은 메시지를 다시 읽음	처리 속도를 PG 용량에 맞추는 백프레셔다

마지막 줄은 팀원 PR(#237)이 먼저 넣은 규칙이다. 두 PR이 같은 파일을 두고 충돌해서, 공통 처리기에 결제 설정이 이 규칙만 더하는 방식(`setBackOffFunction`)으로 옮겼다. 그러면서 "허가 거부는 10번을 넘겨도 DLQ로 가지 않는다"는 테스트를 더했다. 코드 위치: `common.messaging.consumer`의

`ConsumerErrorHandlers`, `ConsumerContainerFactories`.

보관 정리: 지워도 중복이 안 생기게 (#251, ISE-146)

- **무엇을 지우나.** 보낸 Outbox 행, 처리 이력(`processed_events`), 끝난 멱등 기록이다. 두면 테이블이 끝없이 커진다.
- **지운 뒤 같은 이벤트가 다시 오면.** 처리 이력은 막지 못한다. 그래도 중복 환불이 없는 것은 환불 테이블의 `UNIQUE(payment_id)`와 조건부 UPDATE가 마지막 방어선이기 때문이다. 그래도 이력 보관 기간은 토픽 보관 기간보다 길어야 하고, 짧게 설정하면 서비스가 기동을 거부한다.
- **지우지 않는 것.** 미확정 결제(`REQUESTED`·`UNKNOWN`·NMC)의 멱등 기록은 남긴다. 같은 키로 다시 오면 그 기록으로 기존 결제를 돌려줘야 하기 때문이다.
- **두 인스턴스가 동시에 지우지 않게.** PostgreSQL advisory lock을 배치마다 트랜잭션 단위(`pg_try_advisory_xact_lock`)로 건다. 세션 잠금은 연결을 계속 쥐어야 해서 커넥션 풀과 맞지 않는다. 트랜잭션 잠금은 커밋하면 저절로 풀린다.

관측 (ISE-41)

결제는 에러율보다 "돈이 확정되지 않은 채 남은 건과 그 나이"를 먼저 본다(ADR 0024). 그 기준으로 지표를 만들고(#256, ISE-156), 알림 21개와 대시보드를 연결했다(#257, ISE-157). compose에서 Kafka를 멈추자 5분 9초 뒤 `PaymentOutboxDelayed`가 울렸고, 다시 켜자 50초 뒤 풀렸다.

지표의 개념

- **Counter·Gauge·Timer.** Counter는 늘기만 하는 누적 값(실패 수), Gauge는 지금 값(대기 건수, 최고령), Timer는 걸린 시간의 분포다. p95·p99 같은 백분위는 히스토그램 버킷을 켜야 Prometheus가 계산할 수 있다.
- **Prometheus는 당겨 간다(pull).** 서비스가 `/actuator/prometheus`에 값을 내놓으면 Prometheus가 몇 초마다 긁어 간다. 그래서 잠깐 떴다 사라지는 운영자 명령 프로세스의 Counter는 긁힐 틈이 없다. 명령 실행 수는 떠 있는 서비스가 감사 테이블을 세어 Gauge(`payment_ops_actions`)로 낸다.
- **카디널리티.** 라벨 값 조합마다 시계열이 하나씩 생긴다. `paymentId`처럼 끝없이 늘어나는 값을 라벨에 넣으면 Prometheus 메모리가 터진다. 라벨 값을 enum으로만 받게 해 구조적으로 막았다.
- **0으로 미리 등록.** 처음 일어난 사건은 시계열이 없다가 생긴다. 그러면 `increase()`가 "0에서 1로 늘었다"를 못 본다. 값이 정해진 라벨 조합은 기동 때 0으로 만들어 둔다.
- **DB 집계 Gauge.** "운영자 확인이 필요한 결제 수"처럼 DB를 세야 하는 값은 수집할 때마다 쿼리하지 않는다. 15초마다 집계해 메모리에 두고 Gauge는 그 값을 읽는다. 갱신이 멈추면 값이 굳으므로 "배경 작업 정체" 알림이 함께 있어야 한다.

코드 위치: `PaymentMetrics`, `MeteredPgClient`(PG 호출을 감싼 데코레이터), `DbAggregateGauges`. PG 보호 장치(회로 차단기·동시 호출 제한)는 Resilience4j가 제공하는 지표를 그대로 켰다.

알림 규칙 21개

알림은 두 묶음이다. 돈이 확정되지 않았거나 추적이 끊긴 상태는 1건이라도 바로 부른다. 자동 복구가 진행 중이지만 늦어지는 상태는 기준을 넘을 때 부른다.

즉시 확인 7개 (모두 critical)

알림	뜻
`PaymentNeedsManualCheck`	운영자 확인이 필요한 결제가 1건 이상
`RefundNeedsManualCheck`	운영자 확인이 필요한 환불이 1건 이상
`PaymentPgAmountMismatch`	지난 5분 안에 PG 금액과 우리 금액이 다른 경우가 있었음
`PaymentDeadLetterOpen`	DLQ에서 처리하지 못한 메시지가 남아 있음
`PaymentDlqHandlerFailing`	DLQ 처리기가 지난 5분 안에 실패함
`PaymentDlqConsumerLag`	DLQ에 읽지 않은 메시지가 5분 넘게 쌓임
`PaymentOutboxFailed`	영구 오류로 멈춘 이벤트가 있음

기준 초과 14개 (warning·critical을 따로 셈)

알림	warning	critical
`PaymentManualCheckAging` (가장 오래된 결제 NMC)	30분	24시간
`RefundManualCheckAging` (가장 오래된 환불 NMC)	30분	24시간
`RefundPendingAging` (진행 중인 환불)	30분	2시간
`PaymentOutboxDelayed` (안 나간 이벤트)	5분	30분
`PaymentRequestedStuck` (`REQUESTED` 에 머묾러 있는 결제)	60초	
`PaymentUnknownAging` (`UNKNOWN` 결제)	5분	
`PaymentPgCircuitOpen` (PG 회로 차단기 열림)	열리면	
`PaymentJobStalled` (배경 작업의 마지막 성공이 오래됨)	작업별 기준	
`PaymentOutboxRelayStalled` (릴레이의 마지막 성공)	60초	
`PaymentDbConnectionTimeout` (커넥션 풀에서 연결을 못 빌림)	5분 안 1번 이상	

- **"발생" 알림은 `increase(x[5m]) > 0`.** 지난 5분 안에 Counter가 늘었으면 참이고, 더 늘지 않으면 5분 뒤 저절로 풀린다.
- **"멈춤" 알림은 `time() - 마지막 성공 시각`.** 마지막 성공 시각은 기동 시각으로 시작해야 첫 주기 전에 거짓 알림이 울리지 않는다.
- **알림도 테스트한다.** `promtool test rules`에 가짜 시계열을 주고 "몇 분에 어떤 라벨로 울리는가"를 단정한다. 기준값을 일부러 바꾸면 테스트가 실패하는 것도 확인했다. CI(`prometheus-rules`)에서도 돈다.
- **격은 함정.** 테스트 파일을 규칙 폴더에 두자 Prometheus가 `rules/*.yml` 패턴으로 테스트 파일까지 규칙으로 읽어 기동에 실패했다. 테스트는 `infra/prometheus/tests/`로 옮겼다.

대시보드

Grafana 대시보드 `payment.json`(uid `iseonjwa-payment`)에 패널 30개를 두었다. 맨 위는 지금 울리는 결제 알림 표(`ALERTS{service="payment"}`)다. 알림 전송(Alertmanager·메신저)은 두지 않았고, 지금은 Prometheus `/alerts`와 이 표로 본다.

운영자 복구 명령 (ISE-42)

알림을 받은 운영자는 SQL로 상태를 고치지 않고, 명령 6개로만 고친다(#258·#259·#262, ADR 0028). compose에서 "DLQ OPEN 알림 → `dead-letter resolve --no-obligation` → 알림 해제"를 실제로 돌렸다.

왜 직접 SQL을 막나

`UPDATE payments SET status='APPROVED'` 한 줄은 알림을 끄지만 이벤트가 나가지 않는다. 예매는 확정도 환불도 되지 않아 돈이 묶인다. 그래서 상태 변경, 상태 이력, Outbox 이벤트, 감사 기록을 한 트랜잭션으로 묶는 명령만 허용한다.

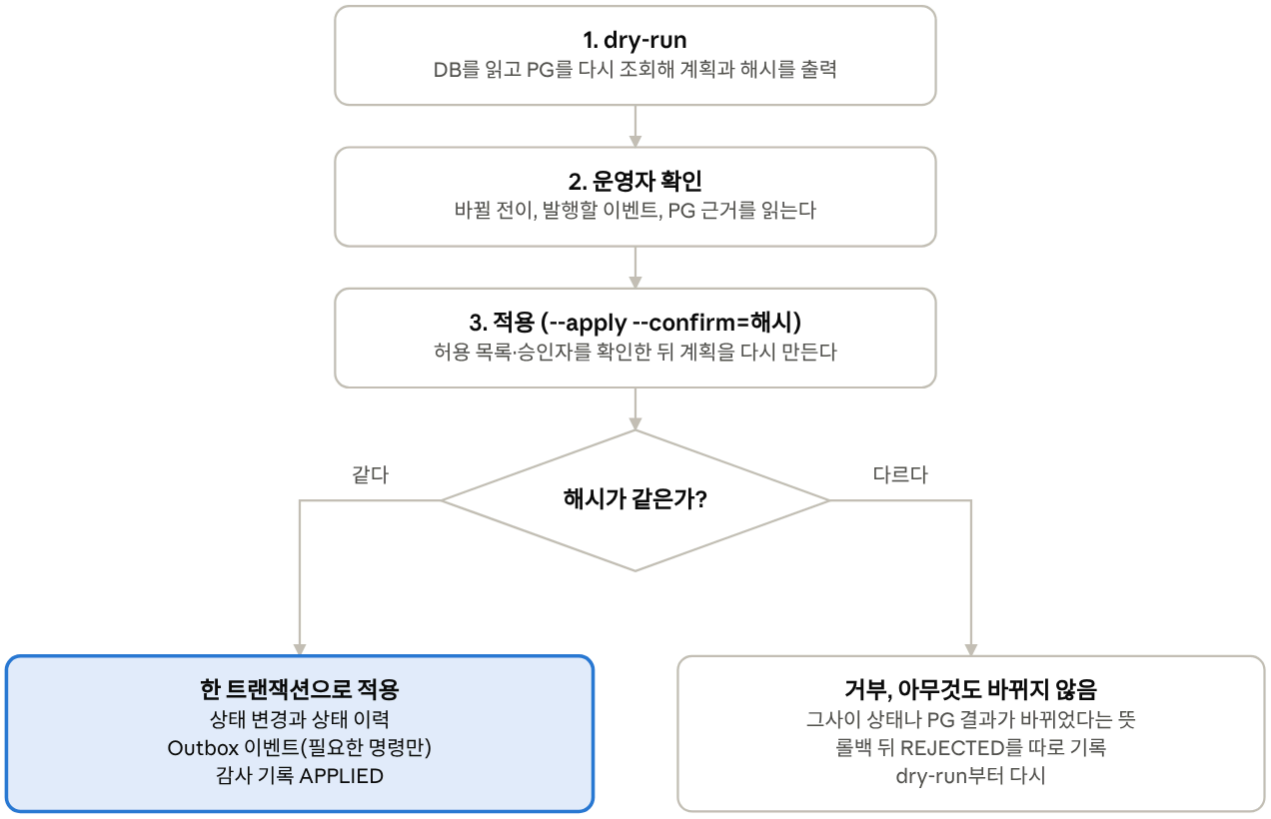
명령 6개

명령	대상	하는 일	PG	4-eyes
`inspect`	결제·환불·예매 ID	상태, 이력, 환불, Outbox, DLQ, 운영 기록을 한 화면에 출력	조회(선택)	아니오
`outbox requeue`	Outbox `FAILED`	원인을 고친 뒤 `PENDING`으로 되돌려 릴레이가 같은 `eventId`로 다시 보냄	없음	아니오
`payment resolve`	결제 NMC	PG 조회 결과로 승인, 거절, 환불 시작(금액 불일치) 중 하나	조회, 늦은 승인 방지	예

명령	대상	하는 일	PG	4-eyes
`refund resolve`	환불 NMC	PG가 환불 완료면 `REFUNDED`. PG가 거절이고 외부 조치 참조가 있으면 `FAILED`로 종결	조회	예
`refund retry`	환불 NMC	PG 장애가 풀린 뒤 `REQUESTED`로 되돌려 환불 워커가 이어받게 함	조회	예
`dead-letter resolve`	DLQ `OPEN`	`--replay`는 DLQ 처리기와 같은 경로로 다시 처리, `--no-obligation`은 환불 의무가 없음을 확인하고 닫음	없음	예

안전장치 다섯 개

1. **목표 상태는 PG가 정한다.** 운영자는 "승인으로 바꿔라"를 고를 수 없다. 명령이 실행 시점에 PG를 다시 조회하고 그 결과에 맞는 전이만 한다. PG를 모르면(`Unknown`) 아무것도 바꾸지 않는다.
2. **미리 보기와 계획 해시.** 기본은 dry-run이다. 무엇이 바뀔지 보여 주고, 대상·현재 상태·PG 근거·계획한 전이의 SHA-256 해시를 준다. 적용할 때(`--apply --confirm=<해시> --reason=<메모>`) 계획을 다시 만들어 해시가 같아야만 실행한다. 확인과 실행 사이에 상태가 바뀌는 문제(TOCTOU)를 막는다.
3. **두 사람 승인(4-eyes).** 돈이나 이벤트가 움직이는 명령은 실행자와 다른 승인자가 있어야 하고, 둘 다 설정의 허용 목록에 있어야 한다.
4. **지울 수 없는 감사 기록.** 성공(APPLIED)은 상태 변경과 같은 트랜잭션에, 거부(REJECTED)는 롤백 뒤 따로 쓴다. 감사 테이블(`manual_action_logs`)과 상태 이력은 DB 트리거가 UPDATE·DELETE·TRUNCATE를 거부한다.
5. **한 번 뜨고 끝나는 JVM.** 같은 이미지를 `ops` 프로필로 띄워 명령 하나만 실행하고 끝난다. 웹 서버, Kafka 소비자, 배경 작업, Eureka 등록을 모두 끈다. 운영용 HTTP 엔드포인트(공격 표면)를 만들지 않기 위해서다.



운영자 명령 적용 흐름 · 3단계, 판단 1개

확인과 실행 사이에 상태나 PG 결과가 바뀌면 해시가 달라져 거부된다. 그래서 운영자는 늘 지금 상태를 기준으로만 고치게 된다.

기억할 만한 설계 결정

- **`refund retry`는 `pg_requested_at`을 비우지 않는다.** 그러면 환불 워커가 바로 환불하지 않고 PG 조회(`findRefund`)부터 다시 시작한다. 이중 환불 방어가 PG 멱등키 하나에만 기대지 않게 하려는 것이다. 설계 문서의 "미확인" 항목을 이 결정으로 닫았다.
- **금액 불일치는 승인하지 않고 환불한다.** PG가 승인한 금액이 우리 금액과 다르면 `payment resolve`는 PG 승인 금액 전액으로 환불을 시작한다. 틀린 금액으로 예매를 확정하지 않는다.
- **확인 못 하는 것은 거부한다.** PG 콘솔에서 직접 한 환불은 우리 환불 ID가 없어 지금 PG 포트로 확인할 수 없다. 그래서 `refund resolve`는 거부하고 NMC로 남긴다. 확인한 척 닫지 않는다.
- **실행 수 지표 이름을 바꿨다.** ADR은 Counter를 가정했지만 일회성 JVM의 Counter는 수집되지 않는다. 감사 테이블을 세는 Gauge `payment_ops_actions`로 바꾸고 ADR 0028 개정 이력에 적었다.

장애 시나리오 테스트 (ISE-44)

장애 시나리오 24개(FT-1~FT-6)를 실제 Mock PG 프로세스와 테스트 PostgreSQL로 돌렸고, 모두 끝에 V1~V3를 통과했다. 검증기 자체 테스트 9개와 함께 3회 반복해도 간헐 실패가 없었다 (#263·#264·#266).

테스트 방법의 개념

- **불변식으로 검사한다.** 장애마다 결과를 따로 단정하는 데 더해, 끝날 때마다 같은 세 가지(V1~V3)를 검사한다. 검증기(`ScenarioInvariants`)는 위반을 모아 한 번에 보여 준다.
- **저장 횟수가 아니라 호출 횟수를 본다.** PG가 멱등이면 같은 주문을 두 번 보내도 저장은 한 번이다. 그래서 "중복 승인 없음"은 호출 수(`approveCalls`)로 봐야 한다. 이를 위해 Mock PG에 호출 수 기록과 원장 조회를 더했다(#263, ISE-128).
- **장애 대상을 좁힌다.** 배경 작업도 PG를 부르므로 "다음 호출 한 번 실패"를 걸면 엉뚱한 호출이 그 한 번을 가져갈 수 있다. 장애는 `orderId` · `refundId`로 한정해 건다.
- **재시작은 "재시작 직후 상태"로 만든다.** 테스트가 프로세스를 죽이는 대신, 운영 코드를 실제로 실행하다 `Error`를 던져 끊는다. 그러면 죽었을 때 남는 DB·PG 상태(TX1만 커밋됨, PG만 승인됨, 임대가 남음)가 그대로 남고, 그 상태에서 복구 경로를 돌린다.
- **기다림은 시계를 당겨 없앤다.** 다음 시도 시각과 임대 만료를 과거로 당기고 `sleep`을 쓰지 않는다. 간헐 실패가 없는 이유다.
- **변이 확인.** 운영 코드에 일부러 버그를 넣어 테스트가 실패하는지 본다. 통과하면 그 테스트는 그 경로를 지키지 못한다는 뜻이다.

시나리오별 결과

묶음	만든 장애	확인한 결과
FT-1 중복	같은 키 3회·새 키 2회, 같은 예매 6건 동시, 환불 요청 3중 발행 + DLQ 재전달, Outbox 전송 확인 전 중단	결제 1건· `approveCalls` 1, 500 없음. 환불 1건·PG 환불 1회. Outbox는 같은 `eventId`로 다시 나가 `SENT` 1행
FT-2 PG 응답 유실	승인·환불 응답 버림, 승인·환불이 PG에 닿지 않음	응답 유실은 조회로 `APPROVED` · `REFUNDED`. 미도달 환불은 같은 `pg_refund_id`로 재호출. 미도달 승인은 늦은 승인 방지 뒤 `DECLINED(NOT_SENT)`, 그 뒤 승인 시도는 PG가 거절
FT-3 재시작	TX1 뒤, PG 승인 뒤, 환불 워커 중간, Outbox 임대 중, DLQ 처리기 트랜잭션 중간	여섯 경우 모두 확정. 임대 중에는 다시 집지 않고, 만료 뒤 조회부터 해서 PG 환불·승인은 1회

묶음	만든 장애	확인한 결과
FT-4 DB 일시 장애	TX1, TX2, 메시지 소비 중, 환불 결과 기록 중에 DB 연결 거절	TX1은 503과 행 없음, 같은 키 재시도 성공. TX2는 202 뒤 재확인으로 `APPROVED`. 나머지도 복구 뒤 환불 1건
FT-5 환불 중단	복구용 회로 차단기 OPEN, PG 환불 장기 장애, PG 환불 거절, PG 콘솔 환불	차단 중에는 시도 수를 올리지 않음. 9회 실패 뒤 NMC → `refund retry` → `REFUNDED`. 거절은 외부 참조가 있어야 `FAILED`로 종결. 콘솔 환불은 거부하고 NMC 유지
FT-6 금액 불일치	PG로 보내는 금액을 1,000원 줄임	승인 API 409, 결제 NMC, 이벤트 없음, 지표 1 증가. `payment resolve`로 PG 승인 금액 49,000원 전액 환불

FT-6은 원래 계획에 없던 시나리오다. ISE-161의 변이 확인에서 "금액 대조를 빼도 아무 시나리오도 실패하지 않는다"가 드러나, 그 경로를 지나는 시나리오를 더했다.

변이 확인 결과

일부러 넣은 버그	잡아낸 시나리오
환불 워커가 조회 없이 바로 환불을 다시 호출	FT-2 (b)·(c), FT-3 (d), FT-4 (d). PG 멱등 덕에 환불 저장은 1건 그대로여서, 호출 수 단정이 차이를 잡았다
회로 차단 중에도 시도 수를 올림	FT-5 (a)
`--close-failed`의 외부 참조 검사를 뺌	FT-5 (c)
TX2의 금액 대조를 뺌	FT-6. DB 제약이 금액이 다른 `APPROVED`를 막아 한 겹 더 있음도 확인했다

DB를 실제로 얼리는 확인은 테스트가 아니라 compose에서 했다(다음 절). 실제 프로세스 강제 종료(`kill -9`)는 하지 못했다. compose의 Mock PG는 장애 주입이 닫혀 있어 "PG 호출 도중"을 만들 수 없기 때문이다.

실제 환경(compose) 확인 결과

테스트 밖에서, 로컬 Docker로 전체 서비스를 띄운 채 네 가지를 직접 확인했다. 세 가지는 설계대로 동작했고, DB 정지 하나가 설계와 다르게 보였다.

확인	한 일	결과
Kafka 정지 중 승인	Kafka 컨테이너를 멈추고 결제 승인	승인 200. Outbox가 5분 동안 13번 재시도, Kafka를 켜자 25초 안에 `SENT`
알림 발화와 해제	실제 Prometheus가 결제 지표를 수집	Kafka 정지 5분 9초 뒤 `PaymentOutboxDelayed` 발화, 다시 켜자 50초 뒤 해제
운영자 명령으로 알림 해제	DLQ OPEN을 만들고 `dead-letter resolve --no-obligation` dry-run → 적용	`PaymentDeadLetterOpen` 발화 후 명령 적용 뒤 해제, 감사 기록 APPLIED
결제 DB 정지	`docker pause`로 결제 Postgres를 얼린 채 승인	설계의 503이 아니라 Gateway 504. DB를 풀자 처리가 이어져 결제 1건 `APPROVED`, 같은 키 재시도는 200으로 같은 결과

DB 정지가 설계와 다르게 보인 이유

테스트의 DB 장애는 "새 연결을 거절"하는 모양이다. 실제로 DB를 얼리면 연결은 살아 있고 쿼리만 멈춘다. 커넥션 풀의 대기 시한(1초)은 빈 연결이 없을 때만 걸리고, 이미 빌린 연결의 쿼리에는 시한이 없다. 그래서 요청은 Gateway 시한(3초)을 넘겨 504가 됐다.

결과는 안전했다. 결제는 1건만 생겼고, 같은 키 재시도는 멱등 기록으로 같은 결과를 받았다(V1 유지). 다만 DB가 멈춘 동안 요청 스레드가 시한 없이 기다린다. 폭주 중이면 스레드가 모두 묶일 수 있다. 그래서 JDBC 소켓·문장 시한을 정하는 후속 ISE-163을 만들었다.

테스트가 통과해도 실제 장애의 모양은 다를 수 있다는 것이 이번 범위에서 가장 큰 발견이다. 테스트는 "그 상태에서 복구가 맞다"까지 보장하고, 장애가 어떤 상태를 만드는지는 실제로 일으켜 봐야 안다.

사고와 판단

사고는 하나였다. AI가 위임받아 머지한 #229가 팀원 PR과 DB 마이그레이션 번호가 겹쳐 dev를 약 10분 깨뜨렸다. 나머지는 진행 중에 내린 판단이다.

Flyway V5 충돌 (TS-31)

- **무슨 일이었나.** #229와 팀원 PR #236이 각자 `V5__...sql`을 더했다. 파일 이름이 달라 Git은 충돌로 보지 않았지만, Flyway는 같은 버전 둘을 거부해 dev에서 결제 서비스가 뜨지 않았다.
- **왜 CI가 못 막았나.** dev 보호 규칙의 필수 검사는 "최신 dev 기준"(strict)이 아니다. PR의 초록 CI가 다른 PR이 머지되기 전 기준일 수 있다.
- **어떻게 고쳤나.** #248(ISE-155)로 내 파일을 V6으로 옮겼고, 뒤따르던 ISE-146은 V7로 옮겼다.

- **그 뒤.** 모든 머지 직전에 최신 dev를 브랜치에 합쳐 다시 테스트하고 마이그레이션 번호를 대조했다. 결제 마이그레이션은 지금 V8까지다.

진행 중 판단

판단	이유
순서를 41 → 44 → 42에서 41 → 42 → 44로 바꿈	환불 중단 시나리오(FT-5)가 운영자 명령 `refund retry`·`refund resolve`를 쓴다
ISE-128(Mock PG 호출 수·원장 조회)을 44보다 먼저	"중복 승인 없음"을 호출 수로 재야 해서 필수 선행이었다. ISE-133에 있던 원장 조회·환불 장애 주입도 이쪽으로 당겼다
ISE-44의 결제·예매 상태 대조를 ISE-43으로 옮김	예매 쪽 내부 조회 API(ISE-75)가 아직 없다. 지금 V3는 "결제 쪽 V3"로 이름을 밝혔다
팀원 PR #237의 백프레셔 규칙을 공통 처리기로 옮김	두 PR이 같은 파일을 두고 충돌했다. 동작은 유지하고 테스트로 고정했다
테스트 DB 연결 한도를 300으로 올림	테스트 컨텍스트가 늘어 연결이 모자랐다. 구현 담당(Sonnet)이 테스트 순서를 고정해 피해 갔는데, 순서 의존은 다른 문제를 가리므로 되돌리고 한도를 올렸다
운영자 명령 실행 수 지표를 Counter에서 Gauge로	일회성 JVM의 Counter는 수집되지 않는다. ADR 0028 개정 이력에 적었다
이미 push한 문서 오류는 새 커밋으로 고침	amend 뒤 force push를 시도했다가 자동 권한 검사에서 막혔다. 팀 규칙상 force push는 사용자 확인 대상이다

팀에 영향이 있는 변경

- Prometheus 컨테이너에 알림 규칙 폴더를 연결하고, 알림 규칙 검사 CI를 더했다(필수 검사는 아니다).
- config-repo 결제 설정에 운영자 허용 목록(팀원 영문 이름 5명)을 넣었다.
- 예매 담당 이슈 ISE-73에 공통 소비 재시도·DLQ 부품 사용법을 알렸다.

머지 위임은 ISE-42까지로 끝났다. 다음 이슈부터는 다시 직접 머지하는 것으로 메모해 두었다.

한계와 남은 일

결제 운영 신뢰성의 남은 큰 일은 결제·예매 대사(ISE-43)와 DB 멈춤 시한(ISE-163) 둘이다. 나머지는 문서에 한계로 적어 두었다.

다음 후보

이슈	내용	먼저 필요한 것
ISE-43	결제·예매 대사. V3의 예매 쪽 절반(결제·예매 상태 쌍 대조) 포함	예매 내부 조회 API(ISE-75, 예매 담당)
ISE-163	DB가 멈출 때의 JDBC 소켓·문장 시한	없음
ISE-130 → ISE-133	Mock PG 공통 계약 테스트	ISE-130이 먼저
ISE-142	간헐적으로 실패하는 테스트	없음
ISE-134	시연용 장애 주입 방식	EC2 배포

확인하지 못한 것과 한계

- **콘솔 직접 환불**은 지금 PG 포트로 확인할 수 없다. PG 환불 금액이 다른 경우의 종결 방법도 설계에 없다. 토스 어댑터(M4) 때 다시 본다.
- **Mock PG는 거절한 환불을 저장하지 않는다.** 그래서 `refund resolve --close-failed`로 닫는 경로는 실제 Mock PG로 만들 수 없고, 테스트에서 조회 응답만 바꿔 확인했다.
- **compose의 Mock PG는 `local` 프로필**이라 장애 주입 API가 닫혀 있다. compose에서는 프로세스 강제 종료와 "승인 성공 경로"의 운영자 명령을 만들지 못했다.
- **알림 전송**(Alertmanager·메신저)은 두지 않았다. 지금은 Prometheus 화면과 대시보드 표로 본다.
- **부하 중 비용**(히스토그램 시계열 수와 메모리, 대량 적체에서의 정리·집계 비용)은 재지 않았다.
- **운영자 ID는 자기 신고**다. 감사 기록의 정확성은 배포 호스트의 SSH 접근 통제에 기댄다.

면접·발표에서 설명하는 법

한 문장으로 말하면 이렇다. "장애가 나도 승인·환불이 두 번 일어나거나 사라지지 않게 했고, 그걸 장애 시나리오 24개와 실제 장애 주입으로 확인했습니다."

Q. PG 응답이 유실되면 어떻게 되나요?

결제를 `UNKNOWN`으로 두고 재확인 작업이 PG를 조회해 확정합니다. PG에 주문이 없으면 망취소로 늦은 승인을 막은 뒤에만 거절로 닫습니다. 중복 승인이 없다는 것은 PG 저장 건수가 아니라 승인 호출 횟수로 테스트했습니다.

Q. Kafka가 죽으면 결제는요?

승인은 그대로 성공합니다. 이벤트는 같은 트랜잭션으로 Outbox 테이블에 남고, 릴레이가 지수 백오프에 흔들림을 섞어 재시도합니다. 실제로 Kafka를 멈춰 보니 5분간 13번 재시도했고, 다시 켜자 25초 안에 발행됐습니다.

Q. 중복 환불은 어떻게 막나요?

한 겹에 기대지 않습니다. 처리 이력(`eventId`), 환불 테이블의 `UNIQUE(payment_id)` 와 조건부 UPDATE, PG 멱등키(`pg_refund_id`)가 각각 막습니다. 환불 워커는 중단됐던 환불을 다시 집으면 환불부터 하지 않고 PG 조회부터 합니다. 이 조회를 일부러 빼면 시나리오 4개가 실패하는 것도 확인했습니다.

Q. 운영자가 수동으로 고칠 때 실수는 어떻게 막나요?

운영자는 결과를 고르지 못하고 PG 조회 결과가 정합니다. 먼저 미리 보기로 계획과 해시를 받고, 적용할 때 계획을 다시 만들어 해시가 같아야만 실행합니다. 돈이 움직이는 명령은 두 사람이 필요하고, 모든 적용은 지울 수 없는 감사 테이블에 남습니다.

Q. 장애 대응을 어떻게 검증했나요?

시나리오마다 끝에 같은 불변식 세 개(승인 중복 없음, 환불 중복 없음, 사라지는 돈 없음)를 검사했습니다. 운영 코드에 일부러 버그를 넣어 테스트가 잡는지도 봤고, 그 과정에서 금액 불일치 경로를 아무 시나리오도 지나지 않는 빈틈을 찾아 메웠습니다.

Q. 가장 크게 배운 점은요?

테스트의 장애와 실제 장애의 모양이 다르다는 것입니다. 테스트에서 DB 장애는 연결 거절이었는데, 실제로 DB를 얼리자 쿼리가 시한 없이 멈춰 503이 아니라 504가 났습니다. 데이터는 안전했지만 스레드가 묶일 수 있어서, 커넥션 풀 시한과 쿼리 시한이 다르다는 것을 배우고 후속 과제로 남겼습니다.

기록: `docs/learning/junhyun/daily/2026-11-03.md` , `docs/ai-judgments/junhyun/daily/2026-11-03.md` , `docs/troubleshooting.md` TS-31